

Análise da Rede de Colaboração do Repositório *Plane* utilizando Estruturas de Grafos

Arthur Modesto Couto
Bernardo Carvalho Denucci Mercado
Matheus Henrique Tavares Malta Soares

¹Curso de Engenharia de Software – Pontifícia Universidade Católica de Minas Gerais
(PUC Minas)
Belo Horizonte – MG – Brasil

{amcouth, bcdmercado, Matheus.soares.1500611}@sga.pucminas.br

Abstract. *This work presents the design, implementation and analysis of a graph-based tool to model the collaboration network of an open-source repository. Using a custom directed and weighted graph structure – implemented from scratch through an abstract API with adjacency-matrix and adjacency-list implementations – interactions among contributors of the makeplane/plane repository (comments, issue closings, pull request reviews and merges) are mined, modeled and aggregated into individual and integrated graphs. Complex-network metrics such as degree, betweenness, closeness and PageRank centralities, as well as density, reciprocity, clustering, assortativity and community detection (Label Propagation) are computed to characterize the collaboration patterns of the project.*

Resumo. *Este trabalho apresenta o projeto, a implementação e a análise de uma ferramenta baseada em grafos para modelar a rede de colaboração de um repositório de código aberto. Utilizando uma estrutura de grafo direcionado e ponderado implementada do zero – por meio de uma API abstrata com implementações em matriz de adjacência e lista de adjacência – são minerados, modelados e agregados as interações entre os colaboradores do repositório makeplane/plane (comentários, fechamentos de issues, revisões e merges de pull requests) em grafos individuais e em um grafo integrado. Métricas de redes complexas, como centralidades de grau, intermediação, proximidade e PageRank, além de densidade, reciprocidade, coeficiente de aglomeração, assortatividade e detecção de comunidades (Label Propagation), são calculadas para caracterizar os padrões de colaboração do projeto.*

1. Introdução

O desenvolvimento de *software* de código aberto depende fortemente da colaboração entre desenvolvedores distribuídos geograficamente, que interagem por meio de *issues* e *pull requests*. Compreender como essa colaboração ocorre – quem são os colaboradores mais centrais, como se formam comunidades de trabalho e qual a coesão geral da rede – é fundamental para avaliar a saúde e a sustentabilidade de um projeto.

Este trabalho tem como objetivo desenvolver uma ferramenta computacional que modele as interações entre colaboradores de um repositório real do GitHub como um

grafo direcionado e ponderado, implementando do zero as estruturas de dados de grafo e os algoritmos de análise de redes complexas exigidos, sem o uso de bibliotecas prontas como NetworkX.

O restante deste documento está organizado da seguinte forma: a Seção 2 apresenta o repositório escolhido e a justificativa dessa escolha; a Seção 3 descreve a estratégia de coleta de dados e a modelagem do grafo (Etapa 1); a Seção 4 descreve a arquitetura e a implementação da ferramenta (Etapa 2); a Seção 5 apresenta a análise dos resultados obtidos a partir das métricas de redes complexas (Etapa 3); a Seção 6 apresenta a interface de visualização desenvolvida; e a Seção 7 apresenta as considerações finais e os trabalhos futuros.

2. Repositório Analisado e Justificativa

O repositório selecionado para este trabalho foi o `makeplane/plane`¹, projeto de código aberto que implementa uma plataforma de gerenciamento de projetos (alternativa de código aberto a ferramentas como Jira, Linear, Monday e ClickUp), atualmente com mais de 46.000 estrelas e mais de uma centena de colaboradores ativos. Trata-se de um dos projetos open source de gerenciamento de projetos mais populares do GitHub, com crescimento expressivo de estrelas e contribuições em curto espaço de tempo.

Devido a esse crescimento acelerado e à grande quantidade de *issues* e *pull requests* abertos diariamente por uma comunidade ativa e diversificada de colaboradores, o repositório oferece uma base rica e significativa de interações (comentários, revisões, aprovações, fechamentos e *merges*), adequada para a modelagem de uma rede de colaboração complexa, conforme exigido pelo enunciado (repositórios com mais de 5.000 estrelas).

3. Modelagem do Grafo (Etapa 1)

3.1. Estratégia de Coleta de Dados

A mineração dos dados foi realizada por meio da API oficial do GitHub (REST, via biblioteca `PyGithub`), priorizando a extração de *issues* e *pull requests* e de suas respectivas interações: comentários, fechamentos, revisões, aprovações e *merges*.

As interações foram mapeadas da seguinte forma:

- O **autor original** da *issue/pull request* é considerado o **destino** (alvo) da interação;
- Os **comentaristas e revisores** são a **origem** da interação, ou seja, a aresta direcionada aponta do colaborador que interagiu para o autor original;
- Robôs (como `dependabot` e `github-actions`) são filtrados por padrão, de modo a não poluir a análise com comportamentos automatizados que distorcem as métricas reais de colaboração humana (essa filtragem pode ser desativada via parâmetro de linha de comando).

A corretude dessa etapa de mineração e construção do grafo é validada pela suíte `tests/test_all.py`, que cobre, de forma independente da API do GitHub (utilizando massas de dados controladas/*mock*): (i) a criação das pastas padrão de dados

¹Disponível em: <https://github.com/makeplane/plane>.

(`data/raw`, `data/processed` e `data/outputs`); (ii) a estrutura dos dados minerados (presença das chaves `repository`, `issues` e `prs`, e dos campos de cada *issue*/PR, como autor e comentários); (iii) a construção correta do grafo a partir dos dados minerados, verificando que usuários do tipo Bot (como `dependabot`) são excluídos quando `exclude_bots=True`, que os pesos de vértice refletem corretamente o número de contribuições de cada usuário, e que os pesos de aresta correspondem à soma dos pesos das interações definidos na Tabela 1 (por exemplo, uma aresta resultante de um comentário mais uma revisão deve ter peso $2 + 4 = 6$); e (iv) a geração física dos arquivos exportados (`.gexf`, `.json` e `.csv`), garantindo que o JSON exportado seja válido e contenha as chaves `nodes` e `links`.

3.2. Grafos Individuais

Conforme exigido na primeira etapa de modelagem, foram construídos três grafos simples e direcionados, cada um representando um tipo específico de relação entre colaboradores:

- **Grafo 1 – Comentários:** arestas representando comentários em *issues* ou *pull requests*;
- **Grafo 2 – Fechamentos:** arestas representando o fechamento de uma *issue* por um usuário diferente do autor;
- **Grafo 3 – Revisões/Aprovações/Merges:** arestas representando revisões, aprovações e *merges* de *pull requests*.

Em todos os casos, caso exista interação bidirecional entre dois colaboradores (por exemplo, *A* comenta em um *post* de *B* e *B* comenta em um *post* de *A*), são utilizadas arestas anti-paralelas, preservando a propriedade de grafo simples (sem laços nem arestas múltiplas).

3.3. Grafo Integrado e Pesos

Além dos grafos individuais, foi construído um **grafo integrado**, no qual cada usuário é representado por um nó e cada aresta direcionada $A \rightarrow B$ representa a soma ponderada de todas as interações em que o colaborador *A* interagiu com conteúdo de autoria do colaborador *B*. A Tabela 1 apresenta os pesos atribuídos a cada tipo de interação.

Tabela 1. Pesos atribuídos a cada tipo de interação no grafo integrado

Tipo de Interação	Peso	Justificativa
Comentário em <i>issue</i> ou <i>PR</i>	2	Interação leve, tira-dúvidas ou sugestão
Abertura de <i>issue</i> comentada por outro usuário	3	Criação de pauta ou <i>bug report</i> qualificado
Revisão/aprovação de <i>pull request</i>	4	Análise técnica profunda do código de outro desenvolvedor
<i>Merge</i> de <i>pull request</i>	5	Validação final de integração (decisão do mantenedor)

Quando um mesmo par ordenado de colaboradores apresenta múltiplas interações de tipos diferentes, os pesos correspondentes são somados. Por exemplo, se o colaborador

A revisou um *pull request* de *B* (peso 4) e também comentou em uma *issue* de *B* (peso 2), a aresta direcionada $A \rightarrow B$ terá peso total igual a 6.

4. Arquitetura e Implementação da Ferramenta (Etapa 2)

A ferramenta foi implementada em Python, seguindo princípios de orientação a objetos, abstração e herança, conforme exigido pelo trabalho.

4.1. Estrutura de Classes

Foi definida uma classe abstrata *AbstractGraph*, responsável por especificar a API comum a todas as implementações de grafo, os atributos compartilhados (rótulos e pesos de vértices, contagem de arestas) e os métodos auxiliares (como a verificação de índices válidos). A partir dela, foram derivadas duas classes concretas:

- *AdjacencyMatrixGraph*(int numVertices): implementação baseada em matriz de adjacência;
- *AdjacencyListGraph*(int numVertices): implementação baseada em listas de adjacência.

A Figura 1 apresenta o diagrama de classes da solução.

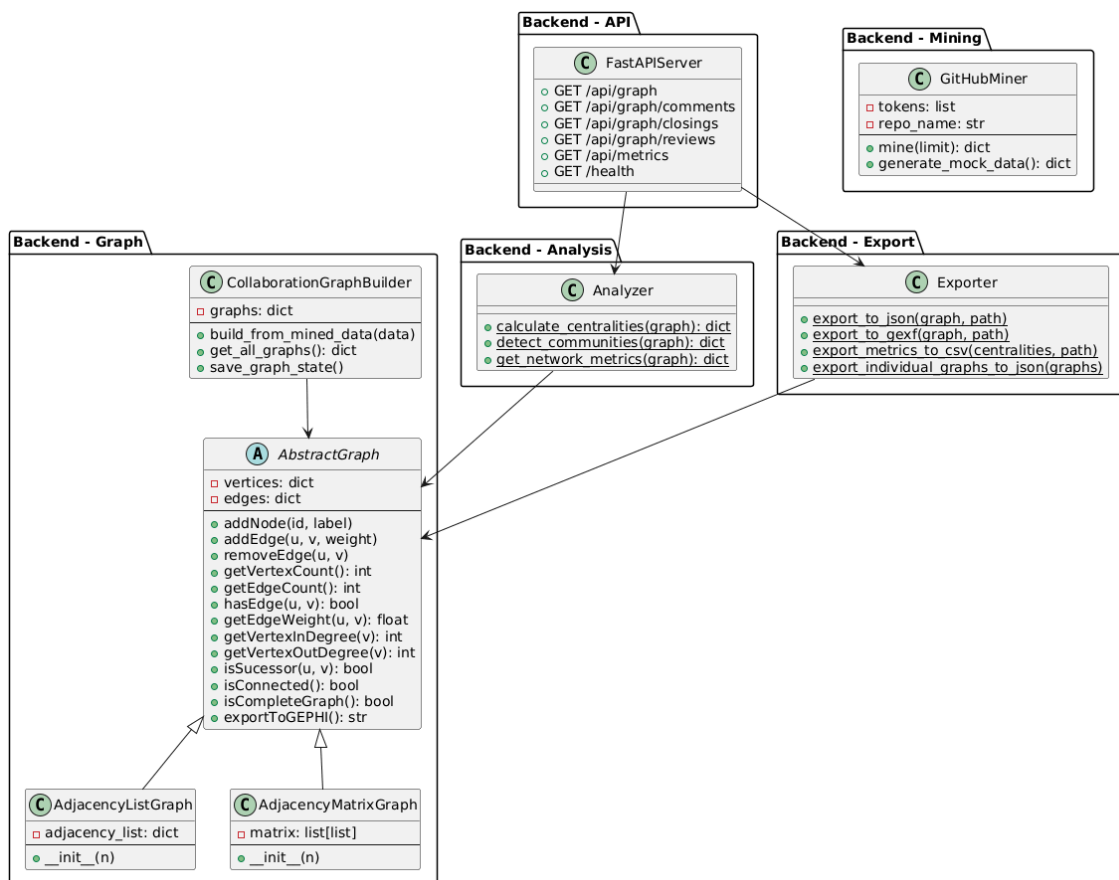


Figura 1. Diagrama de classes da ferramenta de análise de grafos

4.2. API Implementada

A API obrigatória foi implementada em ambas as classes concretas, incluindo: `getVertexCount`, `getEdgeCount`, `hasEdge`, `addEdge`, `removeEdge`, `isSucessor`, `isPredessor`, `isDivergent`, `isConvergent`, `isIncident`, `getVertexInDegree`, `getVertexOutDegree`, `set/getVertexWeight`, `set/getEdgeWeight`, `isConnected`, `isEmptyGraph`, `isCompleteGraph` e `exportToGEPHI`.

As restrições exigidas foram respeitadas: os grafos são simples (não permitem laços nem arestas múltiplas), o método `addEdge` é idempotente, e operações com índices inválidos ou inconsistentes lançam exceções (`IndexError` e `ValueError`).

Embora ambas as implementações estejam disponíveis e tenham sido testadas, a estrutura concreta efetivamente utilizada para representar a rede de colaboração extraída foi a `AdjacencyListGraph`. Essa escolha se deve à esparsidade típica de redes sociais/colaborativas: o grafo integrado do Plane possui 1 496 nós, mas apenas 2 313 arestas – uma densidade de cerca de 0,1% (Seção 5.2). Uma representação em matriz de adjacência exigiria $1496^2 \approx 2,24$ milhões de células, das quais mais de 99,9% estariam vazias, representando um desperdício enorme de memória. A lista de adjacência, por outro lado, armazena apenas as conexões de fato existentes, sendo a escolha natural para grafos com milhares de nós e poucas arestas por nó, como ocorre na grande maioria dos desenvolvedores que interagem apenas com uma pequena minoria de outros colaboradores.

4.3. Pipeline da Aplicação

A Figura 2 apresenta o fluxo geral da aplicação, desde a mineração dos dados até a exportação dos resultados, passando pela construção dos grafos, cálculo das métricas de rede e detecção de comunidades.

Uma aplicação de demonstração separada (`demo.py`) consome a API e exercita todas as operações disponíveis, conforme solicitado no enunciado.

4.4. Testes Unitários

A suíte de testes (`tests/`) cobre a inicialização dos grafos, contagem de vértices e arestas, idempotência de `addEdge`, tratamento de exceções para índices inválidos e laços, cálculo de graus de entrada e saída, relações de sucessão/predecessão/divergência/convergência/incidência, verificação de grafo completo e conectividade, atribuição de pesos e rótulos, além da geração correta dos arquivos exportados (`.gexf`, `.json` e `.csv`).

A Tabela 2 apresenta o plano de aceitação da API, relacionando os principais grupos de requisitos da Seção 4.2 aos testes (`tests/test_graph.py`, com 15 casos de teste) que os validam, executados de forma parametrizada para ambas as implementações (`AdjacencyMatrixGraph` e `AdjacencyListGraph`) via `pytest`.

Como mostra a Tabela 2, os 15 casos de teste cobrem praticamente a totalidade dos métodos da API obrigatória em ambas as implementações concretas (`AdjacencyMatrixGraph` e `AdjacencyListGraph`). A Tabela 3 apresenta o resultado da execução de `pytest --cov=src` (57 testes), que reporta uma cobertura

Tabela 2. Plano de aceitação da API de grafos

Requisito	Casos de teste
Inicialização e contagem de vértices/arestas	test_graph_initialization, test_isolated_vertex
Adição/remoção de arestas, idempotência e arestas inexistentes	test_add_edge_and_counts, test_remove_edge, test_remove_nonexistent_edge
Tratamento de exceções (laços e índices inválidos)	test_self_loop_raises_error, test_invalid_vertex_raises_error
Pesos de vértices e arestas (<i>getters/setters</i>)	test_weights, test_edge_weight_default
Graus de entrada/saída	test_degree
Relações isSucessor/isPredessor/isDisjoint/isDivergent/isConvergent/isIncident	test_is_successor, test_is_predecessor, test_is_disjoint, test_is_divergent, test_is_convergent, test_is_incident
isConnected, isEmptyGraph, isCompleteGraph	test_connected_graph, test_disconnected_graph, test_large_complete_graph
Rótulos de vértices (<i>getVertexLabel/setVertexLabel</i>)	test_vertex_labels

geral de 90% sobre os módulos de grafo, mineração, construção do grafo, análise de métricas e exportação descritos na Seção 3.1.

Os módulos centrais para os objetivos do trabalho – as implementações concretas de grafo (`adjacency_list.py`, `adjacency_matrix.py` e `api.py`), o módulo de análise de métricas (`analyzer.py`), o construtor dos quatro grafos (`builder.py`) e o módulo de exportação (`exporter.py`) – apresentam cobertura entre 90% e 100%, validando a correteza das estruturas de dados, da modelagem dos grafos e dos algoritmos de centralidade, estrutura e comunidades implementados do zero. A cobertura mais baixa permanece no módulo `mining/miner.py` (74%): mesmo após a adição de testes que mockam o cliente do GitHub para validar a inicialização, a rotação de *tokens*, o tratamento de *rate limit* e a persistência de *cache/checkpoint*, os trechos restantes correspondem ao processamento detalhado de revisões e *merges* de *pull requests* durante a mineração real (Seção 3.1), que dependem de respostas específicas da API do GitHub e não são totalmente exercitados pelos testes automatizados baseados em *mock*.

5. Análise do Repositório (Etapa 3)

A partir do grafo integrado, foram calculadas as métricas de redes complexas descritas a seguir, todas implementadas do zero.

5.1. Métricas de Centralidade

Foram calculadas as seguintes métricas para cada colaborador (nó) da rede:

- **Grau (*in/out-degree centrality*):** número de conexões diretas de entrada e de saída de cada colaborador;
- **Centralidade de intermediação (*betweenness*):** calculada via algoritmo de Brandes [Brandes 2001], identifica colaboradores que atuam como "pontes" entre diferentes grupos do projeto;

Tabela 3. Cobertura de testes por módulo (pytest --cov=src)

Módulo	Stmts	Miss	Cobertura
analysis/analyzer.py	218	8	96%
config.py	27	3	89%
export/exporter.py	115	0	100%
graph/adjacency_list.py	45	2	96%
graph/adjacency_matrix.py	49	2	96%
graph/api.py	120	12	90%
graph/builder.py	215	14	93%
mining/miner.py	200	53	74%
TOTAL	989	94	90%

- **Centralidade de proximidade (*closeness*)**: calculada segundo Wasserman & Faust [Wasserman and Faust 1994], adequada para redes parcialmente conexas;
- **PageRank ponderado**: calculado via *power iteration* [Page et al. 1999], mede a influência de um colaborador ponderando a importância de quem o referencia.

Tabela 4. Top 10 colaboradores por PageRank no grafo integrado

#	Usuário	Contrib.	In-Degree	Betweenness	PageRank	Comunidade
1	nickproud	2	0.0120	0.0000	0.00196	0
2	mnbro	5	0.0047	0.0000	0.00188	0
3	LarsJaensch	3	0.0107	0.0000	0.00184	0
4	ahmed-fawzy99	1	0.0107	0.0000	0.00144	52
5	justkelvin	1	0.0047	0.0000	0.00138	0
6	PhilipJonasFranz	1	0.0087	0.0000	0.00123	0
7	leandrogrupozeni	15	0.0080	0.0039	0.00105	0
8	ChandeepKhosa	6	0.0067	0.0000	0.00099	0
9	GravityShaun	2	0.0067	0.0000	0.00097	0
10	leedsjb	62	0.0120	0.0006	0.00097	0

5.2. Métricas de Estrutura e Coesão

As seguintes métricas globais foram calculadas para caracterizar a estrutura da rede como um todo:

- **Densidade**: razão entre o número de arestas existentes e o número máximo possível de arestas ($E/(V \cdot (V - 1))$) para grafos direcionados simples);
- **Reciprocidade**: fração de arestas que possuem aresta reversa correspondente;
- **Coefficiente de aglomeração (*clustering coefficient*) e Assortatividade** [Newman 2003]: medem, respectivamente, a tendência de colaboradores formarem grupos fortemente conectados e a tendência de colaboradores com muitas conexões se conectarem entre si;
- **Componentes conexos fracos e fortes**: calculados via busca em largura (não direcionada) e via algoritmo de Kosaraju [Cormen et al. 2009], respectivamente;

Tabela 5. Métricas estruturais globais da rede de colaboração

Métrica	Valor
Número de nós (V)	1 496
Número de arestas (E)	2 313
Densidade	0,00103 (cerca de 0,1%)
Reciprocidade	0,051 (cerca de 5,1%)
Coeficiente de aglomeração médio	0,126
Assortatividade de grau	-0,035
Componentes fracamente conexos	159
Componentes fortemente conexos	1 453
Diâmetro (rede direcionada)	10

- **Diâmetro da rede:** maior distância (em arestas) entre dois nós quaisquer, calculado quando a rede é fracamente conexa.

5.3. Detecção de Comunidades

A detecção de comunidades foi realizada por meio do algoritmo de *Label Propagation* (propagação de rótulos) [Raghavan et al. 2007], implementado do zero, que permite identificar grupos de colaboradores que trabalham mais frequentemente juntos – representando, por exemplo, times informais dentro do projeto. Também foram analisados os colaboradores que atuam como elos (*bridging ties*) entre diferentes comunidades.

O algoritmo identificou 168 comunidades no grafo integrado. A grande maioria dos 1 496 colaboradores (1 198 nós, cerca de 80% da rede) foi atribuída a uma única comunidade dominante (comunidade 0), formada pelo núcleo de mantenedores e colaboradores recorrentes do Plane. As demais 167 comunidades são, em sua maioria, pequenos grupos de 1 a 35 colaboradores (por exemplo, a comunidade 5, com 35 membros, e a comunidade 52, com 15 membros), representando núcleos isolados de interação – como autores de *issues* pontuais que só interagiram entre si e com poucos mantenedores específicos.

Quanto aos *bridging ties*, os colaboradores com maior centralidade de intermediação (sriramveeraghanta, iam-vipin, hpflatorre, aaryan610 e pushya22, todos pertencentes à comunidade 0) atuam como pontes que conectam contribuidores externos e comunidades menores ao núcleo principal de desenvolvimento, sendo fundamentais para o fluxo de informação entre os diferentes grupos do projeto.

As Figuras 3a a 3d apresentam as visualizações geradas no Gephi [Bastian et al. 2009] para os quatro grafos descritos na Seção 3: os três grafos individuais (comentários, fechamentos de *issues* e revisões/aprovações/*merges* de PRs) e o grafo integrado e ponderado, com as comunidades destacadas por cor.

5.4. Discussão dos Resultados

Os resultados obtidos revelam um padrão típico de redes de colaboração de projetos open source de grande escala. A densidade extremamente baixa (cerca de 0,1%) mostra que, apesar dos 1 496 colaboradores identificados, cada um interage diretamente

com um número muito pequeno de outros participantes – comportamento esperado, já que a maioria dos usuários contribui de forma pontual (abrindo poucas *issues* ou PRs) e interage principalmente com a equipe de mantenedores.

A baixa reciprocidade (cerca de 5,1%) reforça essa assimetria: a maior parte das interações ocorre em um único sentido (um colaborador externo comenta/revisa/encerra algo de autoria de um mantenedor), sem que o mantenedor retribua a interação em sentido oposto com a mesma frequência. Já o coeficiente de aglomeração médio relativamente alto para uma rede tão esparsa ($\approx 0,126$) indica que, localmente, os colaboradores que interagem com um mesmo usuário tendem também a interagir entre si – consistente com discussões em grupo em *issues* e revisões colaborativas de PRs.

A assortatividade de grau levemente negativa ($\approx -0,035$) sugere uma leve tendência *dissortativa*: colaboradores com muitas conexões (mantenedores) tendem a se conectar mais com colaboradores de poucas conexões (contribuidores ocasionais) do que entre si – um padrão clássico de rede *hub-and-spoke*, em que um pequeno núcleo de mantenedores concentra a maior parte das interações com a periferia.

A análise dos componentes confirma essa estrutura: embora existam 159 componentes fracamente conexos (muitos deles formados por usuários isolados que interagiram apenas entre si, sem conexão com o núcleo principal), o maior componente concentra 1 290 dos 1 496 nós (cerca de 86% da rede) e possui diâmetro 10, indicando que, mesmo no núcleo principal, a informação pode levar até 10 “saltos” para se propagar de um colaborador a outro. O elevado número de componentes fortemente conexos (1 453, próximo do total de nós) é esperado em um grafo direcionado dominado por interações assimétricas (contribuidor $\rightarrow$ mantenedor), nas quais raramente existe um caminho de volta.

Por fim, a forte concentração em uma única comunidade dominante (80% dos nós) e a presença de poucos colaboradores com alta centralidade de intermediação reforçam a leitura de que o projeto Plane possui um núcleo relativamente pequeno e coeso de mantenedores, responsável por integrar e dar vazão a uma periferia ampla e fragmentada de contribuidores ocasionais.

6. Visualização e Demonstração da Ferramenta

Os dados gerados pela ferramenta (grafo com métricas embutidas por nó) são disponibilizados por meio de uma API REST construída com o *framework FastAPI* (end-points `/api/graph` e `/api/metrics`), consumida por um *dashboard* web desenvolvido em React, que permite a exploração visual interativa da rede de colaboração do repositório `makeplane/plane`.

7. Conclusão

Este trabalho apresentou o projeto, a implementação e a análise de uma ferramenta de grafos desenvolvida do zero para modelar a rede de colaboração do repositório `makeplane/plane`. Foram implementadas, a partir de uma API abstrata comum, duas estruturas concretas de grafo (matriz e lista de adjacência), respeitando as restrições de simplicidade, idempotência e tratamento de exceções exigidas, além de uma suíte de testes unitários que valida a correteza da API e dos exportadores.

A partir dos dados minerados da API do GitHub, foram construídos três grafos individuais (comentários, fechamentos de *issues* e revisões/aprovações/*merges* de PRs)

e um grafo integrado e ponderado, sobre o qual foram calculadas, do zero, métricas de centralidade (grau, *betweenness*, *closeness* e *PageRank*), métricas estruturais (densidade, reciprocidade, coeficiente de aglomeração, assortatividade, componentes conexos e diâmetro) e comunidades via *Label Propagation*. Os resultados mostraram uma rede esparsa e fortemente assimétrica, com um pequeno núcleo de mantenedores altamente conectado (concentrado em uma única comunidade dominante, com cerca de 80% dos colaboradores) que serve de elo entre uma periferia ampla e fragmentada de contribuidores ocasionais – um padrão característico de redes *hub-and-spoke* observado em diversos projetos *open source* de grande escala.

Como trabalhos futuros, destacam-se: (i) a implementação de um *pipeline* de Integração Contínua (CI) para execução automática da suíte de testes a cada *push*/PR; (ii) a evolução do *dashboard* web para permitir a exploração interativa completa do grafo gerado; (iii) a análise temporal da rede, comparando *snapshots* da colaboração ao longo do tempo para identificar a evolução das comunidades e da liderança técnica do projeto; e (iv) a extensão do módulo de mineração para suportar outras plataformas, como GitLab, ou logs locais de repositórios Git.

Divisão de Responsabilidades

- **Arthur Modesto Couto:** implementação do módulo de mineração (`src/mining/miner.py`), incluindo extração de *issues*, PRs, comentários e revisões via API do GitHub, sistema de *cache/checkpoint*, rotação automática de *tokens* e geração de dados *mock* para testes; definição da lógica de construção dos quatro grafos (individuais e integrado), pesos das interações e filtragem de bots; exportação e configuração da visualização no Gephi (GEXF, cores por comunidade, layouts) e respectiva documentação; elaboração do relatório em LaTeX (contexto, metodologia, análise dos resultados e conclusões); e colaboração no diagrama de classes.
- **Bernardo Carvalho Denucci Mercado:** desenvolvimento do *frontend* em React (*dashboard* completo, componentes `GraphTabs`, `GraphVisualization` e `DashboardMetrics`, *hook* `useGraphData`, integração com `Sigma.js` e estilização com `Tailwind CSS`); implementação do servidor da API com `FastAPI` (`api_server.py`), incluindo os *endpoints* `/api/graph` e `/api/metrics` e configuração de `CORS`; elaboração dos diagramas de classes (UML) e de fluxo (*activity*) do sistema; produção dos *slides* da apresentação (design, conteúdo técnico do *frontend* e demonstração prática); e parte dos testes unitários (`test_graph.py` e testes dos componentes do *frontend*).
- **Matheus Henrique Tavares Malta Soares:** implementação da API de grafos (`src/graph/api.py` – `AbstractGraph`, `src/graph/adjacency_list.py` e `src/graph/adjacency_matrix.py`), com os métodos abstratos e concretos exigidos; desenvolvimento do módulo de análise e métricas (`src/analysis/analyzer.py`), incluindo as centralidades de grau, *PageRank*, *betweenness* e *closeness*, detecção de comunidades via *Label Propagation* e as métricas globais (densidade, reciprocidade, coeficiente de aglomeração, assortatividade, componentes e diâmetro); implementação do módulo de exportação (`src/export/exporter.py`) para JSON, GEXF e CSV, com validação dos

dados; elaboração da suíte de testes unitários (`tests/test_graph.py` e `tests/test_all.py`) para ambas as implementações de grafo; e implementação da lógica de geração do GEXF e da integração das métricas e atributos visuais nos nós para o Gephi.

Referências

- Bastian, M., Heymann, S., and Jacomy, M. (2009). Gephi: An open source software for exploring and manipulating networks. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 3, pages 361–362.
- Brandes, U. (2001). A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 25(2):163–177.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, 3 edition.
- Newman, M. E. J. (2003). The structure and function of complex networks. *SIAM Review*, 45(2):167–256.
- Page, L., Brin, S., Motwani, R., and Winograd, T. (1999). The pagerank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab.
- Raghavan, U. N., Albert, R., and Kumar, S. (2007). Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E*, 76(3):036106.
- Wasserman, S. and Faust, K. (1994). *Social Network Analysis: Methods and Applications*. Cambridge University Press.

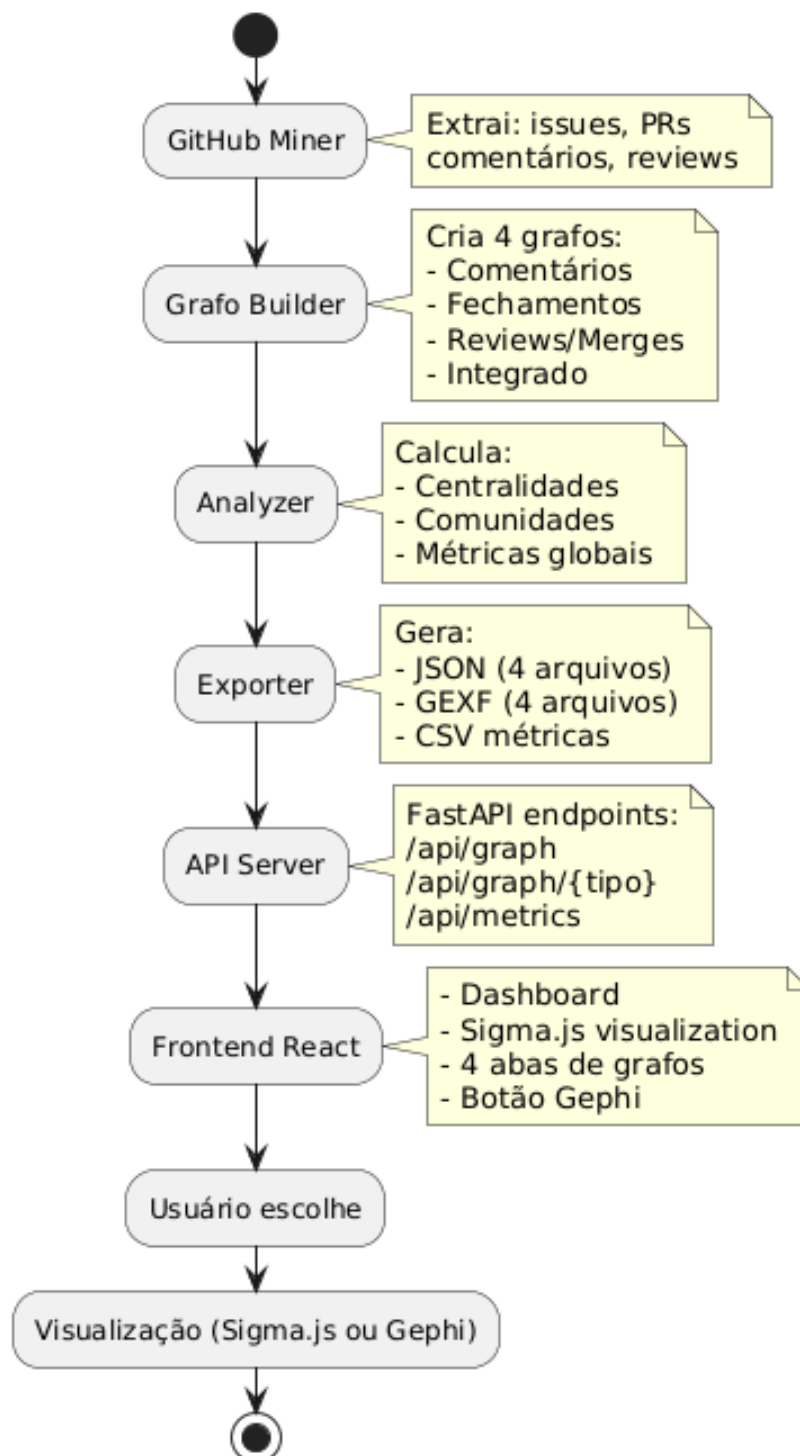
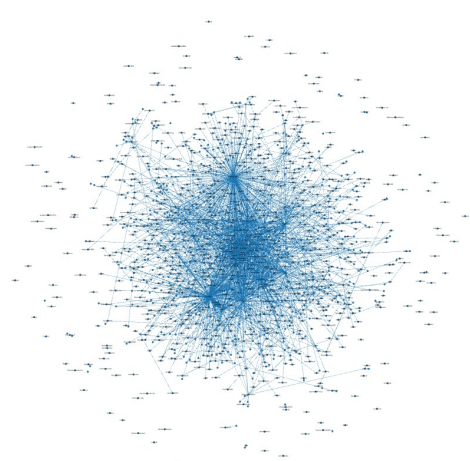
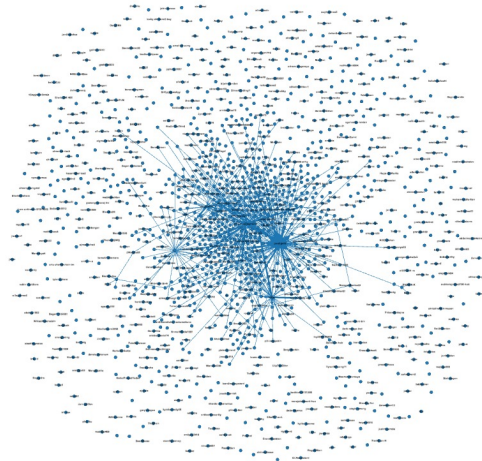


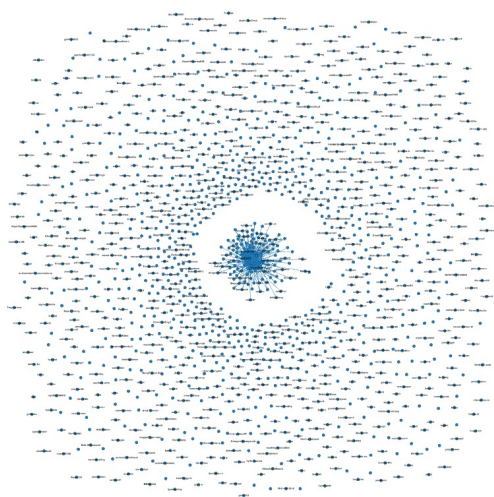
Figura 2. Pipeline de mineração, modelagem e análise da rede de colaboração



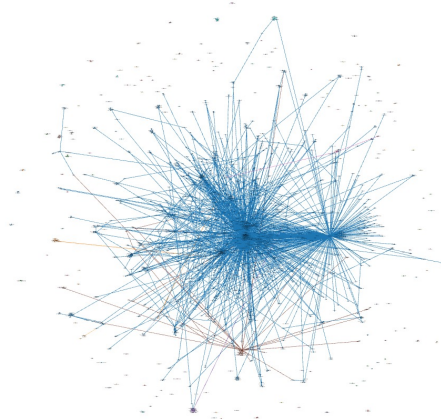
(a) Grafo 1 – Comentários



(b) Grafo 2 – Fechamentos de *issues*



(c) Grafo 3 – Re-
visões/Aprovações/Merges



(d) Grafo Integrado (com comunidades
destacadas por cor)

Figura 3. Visualizações dos grafos de colaboração no Gephi

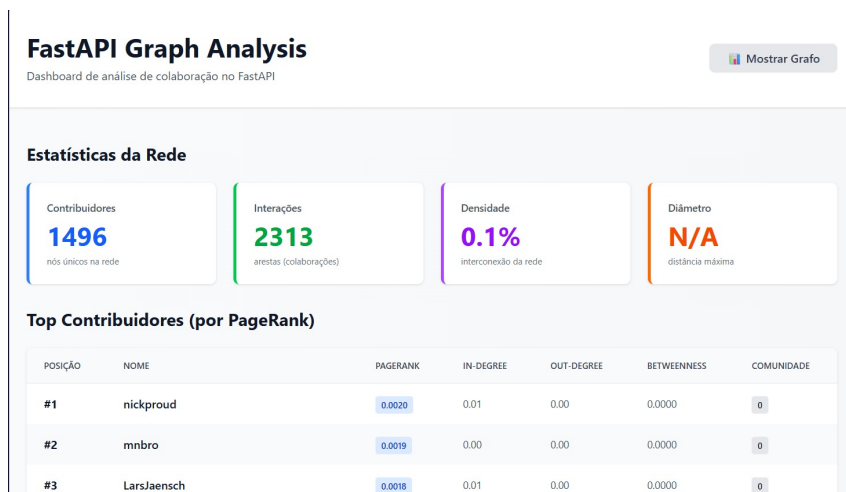


Figura 4. Tela do *dashboard* de métricas da rede de colaboração